

强化

CoEEQA 强化学习部分总结（仅 RL，含具体做法）

1. 一句话结论

这篇 CoEEQA 手稿**适合加入强化学习**，但最好的方式不是简单补一个 PPO，而是把当前的 EQA 流程明确建模为**长时序决策问题**，并对 **controller** 做强化学习微调：学习“何时继续探索、何时调用哪种工具、何时停止并输出答案”。

最推荐的方案是：

> **SFT warm start + 结构化动作空间上的 controller RL + evidence-aware reward**

这样既和现有论文结构自然衔接，又能解释为什么 RL 对 EQA 真正有帮助。

2. 为什么这篇方法适合加 RL

从当前手稿来看，CoEEQA 已经具备了 RL 所需的关键要素：

- 已有明确的 **planner-controller-executor** 三段式 agent 结构；
- controller 本身就是逐步决策模块，负责生成 thought 和 code，决定导航、工具调用与回答时机；
- 已有完整环境反馈：每一步都会产生 observation；
- 已有长轨迹数据：EQA-RT 提供 thought / code / observation 序列；
- 已有探索效率指标：recall@D、epath@D、trajectory success、path length；
- 失败分析已经指出关键瓶颈是：
 - 探索不足；
 - 工具调用并非越多越好，存在冗余调用；
 - 停止时机不理想，容易过早回答。

所以当前方法虽然是 imitation learning / supervised fine-tuning，但本质上已经非常接近一个 POMDP：

- **状态不完全可观测**：只能看到当前视角；
- **动作是多类型的**：导航、检测、裁剪、VQA、回答；
- **回报是长程的**：最终答案对不对，以及探索是否高效。

因此，RL 不是外加模块，而是在补当前 SFT 方法最明显的短板：

> **SFT 会模仿轨迹，但不会直接优化长期探索收益。RL 可以直接优化长期成功率、路径效率和证据充分后的停止策略。**

3. 创新性判断

3.1 创新性不高的版本

如果只是这样写：

- 在 SFT 后追加 PPO；
- 用 final answer correctness 当 reward；
- 把探索写成 MDP；

那么创新性偏低，只能算方法增强，**不太能成为一条独立贡献。**

3.2 创新性较好的版本

如果 RL 是围绕 CoEEQA 当前结构专门设计的，创新性会明显更强。最值得强调的三个点是：

(1) Verifier-guided RL

把现有的 **task verifier / trajectory verifier** 转成 step-level reward source：

- 工具调用是否合法；
- 是否重复调用同一工具；
- 是否在正确时机调用 final answer；
- reasoning / answer 是否存在 hallucination；
- 当前证据是否足以支持最终回答。

这比普通 PPO 更有辨识度，因为它直接复用你现有 pipeline 中的质量控制机制。

(2) Hierarchical RL

和现有 **planner-controller** 结构对齐：

- planner 负责高层子目标；
- controller 负责低层导航与工具调用。

这样 RL 不是附加训练技巧，而是让整个 agent 结构真正变成分层决策系统。

(3) Evidence-aware stopping

把“何时停止探索并回答”作为 RL 的核心学习目标之一，而不只是优化最终答案。

这正对当前 failure mode：

- 探索不足；
- 证据不充分就回答；
- 无效探索过多。

4. 推荐的 RL 设计：只优化 controller，planner 先不动

4.1 为什么主要优化 controller

最推荐的做法是：

- **planner**：保留当前生成整体 plan 的方式，继续作为高层先验；
- **controller**：作为主策略，使用 RL 优化；
- **executor**：作为环境执行器，不参与学习。

原因：

1. 论文当前 controller 已经天然就是逐步决策模块；
2. 失败多来自局部动作选择错误，而不是 plan 完全失效；
3. 直接对 planner 做 RL 代价更高、稳定性更差；
4. 先做 controller RL，最容易在实验中拿到稳定提升。

可在论文中写为：

We formulate the controller as a sequential policy and optimize it with reinforcement learning after supervised warm start, while keeping the planner fixed as a high-level subgoal prior.

5. 具体建模方式

5.1 状态 (State)

定义每一步状态为：

$$s_t = \{Q, o_t, h_t, p, M_t, b_t\}$$

其中：

- Q：问题；
- o_t：当前 observation / 当前视角图像；
- h_t：历史轨迹 (thought, code, observation) ；
- p：planner 给出的整体 plan；
- M_t：显式记忆（已发现对象、已确认属性、已完成工具调用）；

- b_t : 剩余 step budget 或 exploration budget。

5.2 建议额外加入的显式记忆`M\_t`

这是 RL 能否稳定的关键，建议加入 evidence memory：

- 哪些 question-relevant objects 已找到；
- 哪些对象属性已经确认（颜色、大小、位置、状态）；
- 是否已经具备回答问题所需的最小证据集；
- 哪些工具在当前视角上已经试过；
- 是否存在重复调用风险。

这样 reward 就能围绕“证据是否充分”来设计，而不是只围绕最终答案。

6. 动作空间设计（重点）

6.1 推荐做法：结构化动作空间

把 controller 的输出拆成两部分：

一级动作：动作类型

$a_t^{type} \in \{\text{Navigate, ObjectLocation2D, ObjectLocation3D, ObjectCrop, SegmentInstance, VisualQA, FinalAnswer}\}$

二级动作：动作参数

根据动作类型预测参数：

- `Navigate` : `forward / left / right / back / stop`
- `ObjectLocation2D/3D` : 对象名称 + 图像路径
- `ObjectCrop` : `bbox` + 图像路径
- `VisualQA` : 问题模板 + 输入图像
- `FinalAnswer` : 答案文本

6.2 thought 怎么处理

- `thought` 保留为辅助监督目标；
- RL 只优化结构化动作头。

这样你仍能保留论文里“explicit reasoning”的优势，同时避免把 RL 训练难度拉得过高。

7. 奖励函数设计（最核心）

建议总 reward 设计为：

$$R = \lambda_1 r_{ans} + \lambda_2 \sum_t r_{find}^t + \lambda_3 \sum_t r_{info}^t + \lambda_4 \sum_t r_{prog}^t - \lambda_5 \sum_t r_{redund}^t - \lambda_6 r_{prem} - \lambda_7 r_{hall} - \lambda_8 r_{invalid}$$

7.1 最终答案奖励`r\_ans`

当 episode 结束时给予：

- 回答正确： `+1.0`
- 回答错误： `0` 或 `-1.0`

开放词汇任务可直接沿用论文当前的语义一致性评估方式。

7.2 关键对象发现奖励`r\_find`

当**第一次**发现与问题相关的对象时给予奖励：

- 第一次发现第 1 个关键对象： `+0.2`
- 第一次发现第 2 个关键对象： `+0.2`
- 重复看到同一对象： `0`

本质对应当前指标中的 `recall@D`。

7.3 信息增益奖励`r\_info` -> 工具调用正确

当工具调用真正获得新证据时奖励：

- 成功获得目标对象的 3D size；
- 成功获得关键 crop；
- 成功用 VisualQA 确认状态/材质/颜色；
- 从“不足以回答”变为“基本足以回答”。

该项的作用是让模型学会“不是看到了对象就结束，而是要获取能回答问题的证据”。

7.4 路径进展奖励`r\_prog`

鼓励 agent 向尚未发现的目标对象或高价值区域移动。可基于：

- 离下一个目标对象的 geodesic distance 是否缩短；
- 是否更接近 planner 指定的 room / subgoal；
- 是否与 A* shortest path 一致。

如果当前实现上不方便精确算 geodesic，也可以用 A* 轨迹中的 waypoint progress 近似。

7.5 冗余工具惩罚`r\_redund`

用于抑制“工具越调越多”的坏习惯：

- 重复调用同一工具且参数相同：-0.05
- 对同一对象反复定位但没有新增信息：-0.05
- 当前视角下明显不适合调用某工具：-0.05

这和你当前分析中“频繁工具使用并非总是有益”是完全一致的。

7.6 过早回答惩罚`r\_prem`

当还没收集到足够证据就调用 `FinalAnswer` 时给予较大惩罚：

- 缺关键对象；
- 缺关键属性；
- 还未完成必要比较；
- 仅凭语言先验直接回答。

建议设为较强惩罚，例如 -0.3 到 -0.5。

7.7 幻觉惩罚`r\_hall`

用于惩罚以下情况：

- reasoning 中编造未观测到的数值；
- 未调用工具却直接声称知道尺寸、体积、状态；
- answer 无法从已收集 observation 中推出。

这项尤其适合结合现有 trajectory verifier 进行自动判别。

7.8 非法动作惩罚`r\_invalid`

以下情况给予惩罚：

- 工具不在给定 toolset 内；
- 参数格式错误；
- 当前 step 明显不应调用该工具；
- 代码执行失败。

8. 最推荐的训练流程

Stage 1: SFT warm start

沿用当前 CoEEQA 的训练方式：

- 使用 EQA-RT-Train；
- 用高质量 reasoning trajectory 对 controller 做监督学习；
- 保留论文现有的 trajectory sampling 策略。

作用：

- 给 policy 一个强初始化；
- 避免 RL 从随机策略开始，训练太不稳定。

Stage 2: Offline RL

在已有轨迹数据上做离线策略优化。

正样本

- 当前通过 verifier 过滤后的高质量轨迹。

负样本（建议人工构造）

可以从正轨迹扰动生成：

- 提前调用 `FinalAnswer` ；
- 删除某个关键工具调用；
- 重复若干次无效 `GoNextPoint` ；
- 在错误对象上调用定位工具；
- 交换两个关键步骤顺序；
- 用错误答案结束 episode。

Offline RL 的价值

- 充分利用现有 EQA-RT 和 verifier；
- 成本低于直接 online rollout；
- 可先验证 reward 设计是否有效。

Stage 3: Online RL in simulator

在 Habitat / 当前交互环境中真实 rollout：

1. 输入问题和初始位置；
2. planner 生成 plan；
3. controller 依据当前状态选择动作；
4. executor 执行动作并返回 observation；

5. 根据 reward 更新 policy。

推荐算法

首选：

- **PPO**

原因：

- 稳定；
- 易于实现；
- 容易被审稿人接受；
- 对离散动作 + 多步 episode 比较友好。

如需更高级写法，也可以写成：

| PPO-style policy optimization over structured action heads.